

Reviewer Pack — Polymatroid Rank of Fourier Coefficients Bounds Monotone DNF...

Entry 1fdc9129af02 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 22:33:50 UTC

Polymatroid Rank of Fourier Coefficients Bounds Monotone DNF Size for k-CLIQUE

Entry ID: 1fdc9129af02 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 22:33:50 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid Theory **Field B** (complexity object): Monotone DNF Formulas

Statement:

Let $f: \{0,1\}^n \rightarrow \{0,1\}$ be a monotone DNF formula computing k-CLIQUE. Define the polymatroid rank function $\rho(S) = \sum_{i \in S} |\text{FourierCoefficient}(f, i)|$ for subsets $S \subseteq [n]$. Then $\rho([n]) \geq \Omega(n^{\{1/2\}} k^{\{1/4\}})$ and $\rho(S) \leq O(\log n)$ for all S with $|S| \leq \text{poly}(n)$.

Rationale (proposer's reasoning):

Polymatroid rank functions capture submodular structure, which aligns with Fourier coefficients' decay properties. The conjecture links the geometric complexity of Fourier support to monotone circuit size, leveraging the known $\Omega(n^{\{1/2\}} k^{\{1/4\}})$ lower bound for k-CLIQUE.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7045573a2e6621eb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=16.6s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def resolve(cnf):
        n = len(cnf[0])
        dnf = []
        for clause in cnf:
            if not clause:
                return None # Empty clause means unsatisfiable
            new_clause = [x for x in clause if x != -1]
            if not new_clause:
                continue
            dnf.append(new_clause)
        return dnf

    def fast_walsh_hadamard_transform(a):
        n = len(a)
        if n == 1:
            return a
        even = fast_walsh_hadamard_transform(a[::2])
        odd = fast_walsh_hadamard_transform(a[1::2])
        result = [0] * n
        for k in range(n // 2):
            result[k] = even[k] + odd[k]
            result[k + n // 2] = even[k] - odd[k]
        return result

    def fourier_coefficient(f, i):
        n = len(f)
        a = [0] * (1 << n)
        for j in range(1 << n):
            if f[j]:
                a[j ^ i] += 1
        return Fraction(a[0], 2 ** n) + Fraction(a[(1 << n) - 1], 2 ** n)
```

```

def polymatroid_rank(f, S):
    return sum(abs(fourier_coefficient(f, i)) for i in S)

n = 40
k = 5

cnf = []
for _ in range(30):
    clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
    cnf.append(clause)

dnf = resolve(cnf)
if dnf is None:
    return {
        "metric_name": "polymatroid_rank",
        "metric_value": 0,
        "instances_tested": 30,
        "conjecture_holds": False,
        "counterexample": "unsatisfiable CNF"
    }

S = set(range(n))
rho_n = polymatroid_rank(dnf, S)
rho_S = [polymatroid_rank(dnf, {i}) for i in range(1, min(101, n))]

if rho_n < math.sqrt(n) * k ** (1/4):
    return {
        "metric_name": "polymatroid_rank",
        "metric_value": rho_n,
        "instances_tested": 30,
        "conjecture_holds": False,
        "counterexample": f"rho([n]) = {rho_n} < {math.sqrt(n) * k ** (1/4)}"
    }

if any(rho > 10 for rho in rho_S):
    return {
        "metric_name": "polymatroid_rank",
        "metric_value": max(rho_S),
        "instances_tested": 30,
        "conjecture_holds": False,
        "counterexample": f"rho(S) = {max(rho_S)} > 10"
    }

return {
    "metric_name": "polymatroid_rank",
    "metric_value": rho_n,
    "instances_tested": 30,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i + 7 for i in range(5, 8)]

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

metric_values = [r["metric_value"] for r in results if "metric_value" in r]
conjecture_holds = all(r["conjecture_holds"] for r in results if "conjecture_holds" in r)

mean = sum(metric_values) / len(metric_values)
std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if conjecture_holds:
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE no support found")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90377edb.py", line 110, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90377edb.py", line 75, in run_trial
    rho_n = polymatroid_rank(dnf, S)
             ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90377edb.py", line 54, in polymatroid_rank
    return sum(abs(fourier_coefficient(f, i)) for i in S)
               ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90377edb.py", line 54, in <genexpr>
    return sum(abs(fourier_coefficient(f, i)) for i in S)
               ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_90377edb.py", line 49, in fourier_coefficient
    if f[j]:
       ~^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError, preventing data collection. Support condition (80% success) cannot be evaluated without results. | next: Fix index out-of-range error in fourier_coefficient by validating input indices or ensuring proper DNF construction

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136207
2	propose	ollama_remote	qwen3:8b	0	0	53796
3	preregistration	ollama_remote	qwen3:8b	0	0	31883
4	novelty	ollama_remote	qwen3:8b	0	0	27316
5	novelty	ollama_remote	qwen3:8b	0	0	22691
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16796
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12313
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13419
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12711
10	judge	ollama_remote	qwen3:8b	0	0	108396

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 435529 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/1fdc9129af02.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1fdc9129af02.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1fdc9129af02.tar.gz` (if generated)